



Indus OS devices

The Indus Swipe keyboard is not available as a standalone app, here is a list of Indus OS devices <https://digit.in/sep19-49>



Vernacular Digit

Did you know that Digit is available in a number of local languages? Just check the box on the far right <https://digit.in>

design is also an interesting challenge for many Indian languages, since there are lots of different characters that need to be typed. In Gboard, we designed “dynamic” layouts, where you can type a consonant, and the different vowel options appear at the top of the layout. But another very popular input method for Indian languages, and other languages with complex scripts, is to use transliteration, romanizing the native-script characters into the Latin alphabet, often according to their phonetic pronunciations. We use machine learning to let users type in romanized Indian languages, turning the output back into the native script. For example, our transliterated Hindi keyboard allows users to type “daanth” and get “दाँत” (teeth). Now, compared to some languages like Chinese and Japanese, transliterations for Indian languages are more fuzzy; for example “daant” would be a valid alternative for “दाँत”. So we use machine learning to build a model that can map Latin key sequences, like “daant” or “daanth”, and target script symbol sequences across 22 Indian languages.”

BEYOND KEYBOARDS

A keyboard is not strictly necessary to get to the end goal of feeding in text to whatever application is on screen. The Indus Swipe keyboard allows users to enter text by dictating the content. Google ups the ante by providing a handwriting option along with voice typing. Users can simply use their fingers to draw out the words, and these get entered into the application. The feature is available for only a subset of all the languages supported by Gboard.

Daan says, “Beyond typing, many Indian users also frequently use Gboard’s voice typing and handwriting features. These are again examples of how we use machine learning to make it easier to communicate on your smartphone in languages with complex scripts. Speaking of complex scripts, we also had to deal with the fact that some Indian languages have multiple writing systems! Bodo, for

example, can be written in the Bengali or Devanagari scripts.”

Now machine learning is used to recognise this handwriting as well. When asked about how handwriting support for a new language is added to Gboard, Daan replied, “the starting point for the handwriting recognizers in Gboard are the touch points. The drawn input is represented as a sequence of strokes and each of those strokes, in turn, is a sequence of points each with a timestamp attached. Since Gboard is used on a wide variety of devices and screen resolutions our first step is to normalize the touch-point coordinates. The sequence of curves represents the input, but we still need to translate the sequence of input curves to the actual written characters. For that we use a multi-layer recurrent neural network (RNN) to process the sequence of curves and produce an output decoding matrix with a probability distribution over all possible letters for each input curve, denoting what letter is being written as part of that curve. In order to provide the best user-experience, accurate recognition models are not enough — they also need to be fast. To achieve the lowest latency possible in Gboard, we convert our recognition models (trained in TensorFlow) to TensorFlow Lite models. This involves quantizing all our weights during model training such that instead of using four bytes per weight we only use one, which leads to smaller models as well as lower inference times.”

TOWARDS A STANDARD

All these different layouts and changes in the same keyboards over time means that the keypads are inconsistent. Reverie is working towards the solution by developing a standard for Indic keypads. Pani explains, “Indian language keyboards do not have a smartphone for standards at all. If you buy any new phone from any brand, and if you select the keypad, you know exactly how the layout is going to be - it is a QWERTY keypad. You are able to call it QWERTY, and you know that

everybody will understand what it means. So there is a layout of characters, let’s say you have learnt it on one of your phones, and you change your phone, you do not feel alienated by that new keypad on the new phone. It is exactly the same layout. You do not lose anything in terms of how you have got used to typing in English. Whereas a similar thing for Indian languages does not exist. The only standard that existed was the INSCRIPT keypad layout standard that was created for IO1 key computer keyboards in 1988. In 1988 whatever devices that were there, may not be as relevant today. The keypads in mobile phones are much smaller in size, and therefore have fewer keys and all that. Since a standard does not exist for mobile phones, it is obviously difficult for Indians to get used to anything. Every phone may have a different keypad, and different behavior within the keypad. That is the reason why I think it is very critical that there has to be a standard. If I have learned to type, I cannot teach a friend because he may have a different phone model, and I would not know about it. Whereas if there is a standard, the problem is solved.”

All the developers for Indic language keyboards have to work with Unicode, as that is the computing standard used on the OS level. There are a lot of problems with basic input, editing, search, and compatibility across apps. Even using desktop computers, to print the exact representations of the Hindi letters and words that we needed for this article, we had to use the onscreen keyboard in Google Input tools, copy the text to Google Docs, export as PDF, save the PDF as an EPS using Photoshop, copy the letters from the EPS using Illustrator, and paste it into InDesign. Even if the developers have an idea of the shortcomings of a smartphone keyboard, and the necessary features that need to be implemented, they cannot implement it because of the constraints of the Unicode standard. However, that is another story. 